

Scout, Informe metodológico

Competencia "Tu tiempo, tu Mundial" · Facultad de Ingeniería, Universidad Austral · Junio 2026

Integrantes: Julián Ritondale, Fernando Santisi, Juan Decoud

Resumen ejecutivo

Scout es un sistema de recomendación de partidos del Mundial 2026 que clasifica los 72 encuentros de la fase de grupos en tres categorías (**Imperdible**, **Vale la pena**, **Para ver el resumen**) a partir del perfil de cada usuario. El sistema fue diseñado para operar sin datos etiquetados, los partidos aún no se jugaron, lo que hizo inviable el aprendizaje supervisado clásico y motivó una arquitectura bayesiana explicable.

Enfoque central: cada partido se describe con 8 factores normalizados, 4 de ellos no derivables del ranking FIFA (*competitividad*, *grupo de la muerte*, *rivalidad histórica*, *último baile de una leyenda*). Los pesos de esos factores son distribuciones, no números fijos, lo que permite: (1) propagar la incertidumbre al score para guiar la clasificación, (2) aprender del feedback del usuario con un filtro de Kalman preservando lo que el usuario explícitamente declaró, y (3) elicitar el prior desde una descripción en lenguaje natural asistida por LLM.

Resultados: la demo es pública, funciona sin login, integra Google Calendar, muestra la justificación de cada recomendación y reporta la precisión del modelo contra el feedback real del usuario.

1. Problema y objetivo

Dado el **perfil de un usuario** (afinidad + disponibilidad), recomendar qué partidos de la **fase de grupos del Mundial 2026** (72 encuentros) ver, clasificándolos en tres categorías y **justificando** cada decisión:

- **Imperdible:** indispensables por relevancia y afinidad.
- **Vale la pena:** interesantes pero no cruciales, o en horarios complejos.
- **Para ver el resumen:** de bajo interés para el perfil, o en horarios imposibles.

Restricción central: **no hay datos etiquetados** (los partidos no se jugaron y nadie marcó qué es "imperdible"). Esto descarta un modelo supervisado y motiva una **heurística explicable** con una capa **bayesiana** para los pesos y el aprendizaje.

2. Datos y fuentes

Dataset estático, versionado en el repo (frontend/scripts/sources/ → generado a frontend/src/data / por frontend/scripts/build_data.py):

- **Fixture, equipos y sedes:** dataset *openfootball/world-cup* (72 partidos de grupos, 48 selecciones,

16 estadios). Cada partido trae `kickoff_utc` (instante UTC ISO), sede, ciudad y zona horaria.

- **Ranking FIFA y figuras** (hasta 15 por selección, 218 jugadores en total): expandidos y verificados por búsqueda web (mayo–junio 2026), unidos por código FIFA. Incluye plantel completo de los equipos sudamericanos y europeos principales (ARG, BRA, URU, COL, ESP, POR, FRA, ENG, GER, CRO, NED, entre otros).

Validaciones automáticas del generador: 72 partidos (6 por grupo, 24 por jornada), round-robin correcto, integridad referencial, y la regla FIFA de **fecha 3 simultánea** en los 12 grupos.

3. Arquitectura

App **estática client-side** (React + Vite) desplegable en GitHub Pages: todo el motor corre en el navegador sobre los JSON locales, sin backend obligatorio. Un servidor opcional (Express + Gemini) enriquece la redacción de la justificación; si no está, se usa la justificación local (degradación elegante).

4. Variables del perfil (inputs)

Variable	Descripción
Equipos favoritos	Con prioridad (1°, 2°, ...)
Jugadores favoritos	Figuras seguidas (suben sus partidos aunque su selección no sea favorita)
Disponibilidad	Franjas por día (mañana/tarde/noche)
Zona horaria	IANA; default = la del navegador
Tolerancia	Cuánto estira lo "mirable" fuera de las franjas
Perfil fan	casual / total: corre los umbrales de clasificación
Calibración (opcional)	

Importancia 0–100 por factor → fija medias de los priors y aumenta la confianza

5. Features del partido

Ocho factores normalizados a $[0, 1]$. Con $\text{fuerza}(r) = \text{clamp}(1 - (r-1)/50, 0, 1)$ sobre el ranking FIFA r :

Feature	Fórmula
equipo	$1 - 0.15 \cdot (\text{prioridad} - 1)$ si juega un favorito; 0 si no
jugador	$0.7 + 0.3 \cdot (n-1)$ con n = favoritos presentes (0 si ninguno)
estrellas	$(\text{fuerza}(\text{local}) + \text{fuerza}(\text{visitante})) / 2$
competitividad	$\text{clamp}(1 - \text{dist}/30, 0, 1)$, con $\text{dist} = \text{abs}(\text{rank_local} - \text{rank_visitante})$
grupo_muerte	$\text{fuerzaProm} \cdot (0.4 + 0.6 \cdot \text{paridad})$, $\text{paridad} = 1 - (\text{max} - \text{min})$ de fuerzas del grupo
jornada3	jornada 3 → 1; jornada 2 → 0.25; jornada 1 → 0
rivalidad	$\text{max}(\text{storyline curado}, \text{derbi de confederación})$; 0 si ninguno

ultimo_baile	
	intensidad de la leyenda más icónica en (probable) último Mundial; 0 si no juega ninguna

competitividad, grupo_muerte, rivalidad y ultimo_baile son los factores ****no obvios** más allá del ranking FIFA* *que premia la competencia: capturan la paridad (partido emocionante), el drama de grupo*, la emoción del cruce* (rivalidades históricas y derbis regionales) y la *narrativa* de ver a un ídolo despedirse, por encima del nivel absoluto.

Datos curados (frontend/src/data/): rivalidades.ts lista cruces con storyline verificados contra el fixture (p. ej. Francia–Senegal, revancha de 2002; Inglaterra–Croacia, semifinal de 2018) más un piso por derbi de confederación; ultimoBaile.ts lista figuras en probable último Mundial con su intensidad (Messi, Cristiano = 1.0).

6. Modelo de afinidad (bayesiano)

Cada peso es una variable aleatoria $w_k \sim \text{Normal}(\mu_k, \sigma_k^2)$. La afinidad y su incertidumbre se propagan:

```

afinidad  $\mu(m) = \sum_k \mu_k \cdot f_k(m)$ 
varianza  $\text{Var}(m) = \sum_k f_k(m)^2 \cdot \sigma_k^2$ 
incertidumbre(m) =  $\sqrt{\text{Var}(m)}$ 

```

Priors por defecto (medias suman 1 $\rightarrow$ afinidad en [0,1]):

Factor	μ	σ
equipo	0.30	0.04
jugador	0.16	0.05
estrellas	0.14	0.09

competitividad	0.14	0.11
grupo_muerte	0.07	0.10
jornada3	0.07	0.08
rivalidad	0.06	0.10
ultimo_baile	0.06	0.09

σ refleja la **confianza a priori**: alta en lo que el usuario afirma (equipo/jugador), menor en factores inferidos o "no obvios" (competitividad, rivalidad, último baile). **Calibrar** el perfil redefine las medias (normalizadas) y reduce σ a la mitad.

Elicitación del prior. Las medias se elicitán de dos maneras intercambiables: (a) un **cuestionario** de 8 preguntas de un toque que mapea cada respuesta a la importancia de uno o más factores; o (b) **asistida por LLM (opcional): el usuario se describe en lenguaje natural y un LLM** (OpenAI gpt-4o**) traduce esa descripción a: importancia 0–100 por factor + perfil de fan + tolerancia + equipos/jugadores detectados + **franjas de disponibilidad horaria** + lista de métricas no cubiertas. El sistema resuelve automáticamente: apodos ("el bicho" → Cristiano Ronaldo, "la pulga" → Messi), clubs a jugadores del Mundial (River → Montiel, Real Madrid → Vinícius + Mbappé + Bellingham), regiones a selecciones ("sudamericanos" → ARG, BRA, URU, COL, ECU, PAR) y franjas horarias del texto ("a la mañana no puedo" → disponibilidad tarde+noche todos los días). Para las métricas no cubiertas por el texto, el sistema presenta hasta 3 preguntas de complemento dirigidas antes de finalizar. El LLM **no recomienda**: solo convierte texto libre en el prior; todo el razonamiento posterior es del modelo bayesiano explicable. Sin servidor/clave, la app usa solo el cuestionario.

7. Disponibilidad

Se convierte `kickoff_utc` a la zona horaria del usuario (`Intl.DateTimeFormat`, sin dependencias) y se mide el solapamiento con sus franjas:

- Dentro de una franja → **bueno**.

- Fuera, pero a $\leq$ *margen de tolerancia* → **complejo**.
- Más lejos → **imposible**.

Margen por tolerancia: baja = 0 h, media = 1.5 h, alta = 3 h. Sin franjas declaradas, se asume disponibilidad (no penaliza).

8. Clasificación

Combina (μ , σ , encaje). Umbrales por perfil fan: casual {alto 0.6, medio 0.35}, total {alto 0.5, medio 0.3}. Sea $\tau_\sigma = 0.16$ el corte de "score poco confiable" (recalibrado para 8 factores: sumar features independientes eleva el σ agregado del score):

```

si encaje == imposible          → Resumen
si esFavorito:                  # juega su equipo (top-3) o un jugador favorito
    si encaje == bueno          → Imperdible
    si no                      → Vale la pena
si  $\mu \geq$  alto:
    si encaje == bueno y  $\sigma \leq \tau_\sigma$  → Imperdible
    si no                      → Vale la pena
si  $\mu \geq$  medio o  $(\mu + \sigma) \geq$  medio → Vale la pena
si no                          → Resumen

```

Override por favorito (señal dura). Que juegue el equipo o el jugador favorito es, por definición, un partido o de máxima prioridad: la intuición del hincha domina al score lineal, que diluiría ese interés cuando el rival es flojo (p. ej. Argentina vs. un rival de bajo ranking en la fecha 1 no debería caer a "Vale la pena" para un hincha argentino). Por eso un favorito fuerte ($f_{\text{Equipo}} \geq 0.7$, es decir top-3, o $f_{\text{Jugador}} \geq 0.7$) va directo a Imperdible si el horario acompaña, o a Vale la pena si es incómodo.

Para el resto, la **incertidumbre alimenta "Vale la pena"** en dos sentidos: *demote* (afinidad alta pero σ grande no llega a Imperdible) y *promote* (afinidad bajo el corte cuyo límite optimista $\mu + \sigma$ lo alcanza → "apuesta que podría valer", criterio tipo *upper confidence bound*).

9. Aprendizaje por feedback

Cada [+1]/[-1] sobre un partido visto actualiza el **posterior** de los pesos con un update bayesiano conjugado (filtro de Kalman diagonal), con $y = 1/\theta$, ruido de observación $R = 0.25$:

$$\hat{y} = \sum \mu_k f_k ; \quad e = y - \hat{y} ; \quad S = \sum a_k^2 \sigma_k^2 + R$$

$$g_k = \sigma_k^2 a_k / S ; \quad \mu_k \leftarrow \mu_k + g_k \cdot e ; \quad \sigma_k^2 \leftarrow \sigma_k^2 \cdot (1 - g_k a_k)$$

Asignación de crédito (para no castigar lo afirmado ante un [-1] a, p. ej., un partido aburrido del equipo favorito):

1. **σ angosto** en lo afirmado → ganancia chica → casi no se mueve. 2. **Saliencia por sorpresa:** $a_k = \max(0, f_k - f_{-k})$, desviación respecto del promedio de la "dieta" del usuario (sus $K=12$ partidos de mayor afinidad). Lo que está *siempre presente* aporta poca información. 3. **Chip "¿qué no te gustó?":** atribución directa, horario no toca pesos (es disponibilidad); `nivel` ajusta

estrellas/competitividad/grupo/jornada/rivalidad; no me interesaba ajusta equipo/jugador/último-baile.

El posterior se obtiene plegando todo el feedback sobre el prior base (replay), de modo que respeta también la calibración. La UI muestra **qué aprendió** el modelo (qué factores subieron/bajaron).

10. Justificación

- **Local (siempre):** se arma a partir de los factores con mayor contribución y el encaje horario.
- **IA (opcional):** POST /recommendations/justify (server) llama a **OpenAI gpt-4o-mini** con los mismos factores; el prompt incluye restricción explícita de no mencionar jugadores que no jueguen en el partido. Ante error/offline se hace fallback a la local. La demo pública funciona sin servidor.

11. Validación / métrica de éxito

Sin verdad de campo, la validación combina una **métrica de precisión sobre el feedback del usuario** con **consistencia y validez aparente**:

- **Precisión del modelo (implementada y visible en la UI):** para cada partido que el usuario calificó con [+1]/[-1], se mide si el modelo **cold-start** (los priors derivados del onboarding, *antes* de aprender) lo había predicho correctamente (afinidad $\geq$ umbral $\iff$ "le gusta"). Se reporta como **"acertó X de N (Y %)"**. Usar los priors base, no el posterior, evita la fuga de información de medir con los mismos pesos que el feedback ya ajustó; los [-1] por horario se excluyen (son disponibilidad, no afinidad). Implementación en `frontend/src/lib/recommender/accuracy.ts`.
- **Suite de tests** (vitest, 36 casos) que fija el comportamiento esperado: features (incluidos rivalidad y último baile), conversión de zona horaria, umbrales de clasificación, rol de la incertidumbre, asignación de crédito del aprendizaje y el cálculo de precisión.
- **Escenarios de validez aparente:** p. ej. un hincha argentino obtiene los partidos de Argentina como Imperdibles; un partido en horario imposible cae a Resumen.
- **Curva de aprendizaje:** la app expone *qué aprendió* el modelo (movimiento de cada peso con el feedback), de modo que la mejora de la precisión a medida que el usuario califica es observable.

12. Limitaciones y extensiones

- Datos de un evento futuro: ranking y planteles son los mejores disponibles a junio 2026 (218 jugadores verificados); los clubs de los jugadores se actualizan manualmente en el dataset.
- La disponibilidad se evalúa por día/franja; no modela partidos que cruzan medianoche entre días.
- Las rivalidades curadas dependen de que el cruce exista en el fixture de grupos; la cobertura crece con cada storyline agregado.
- **Extensiones:** clubes seguidos, actualizaciones dinámicas de resultados, fases eliminatorias y estudio con usuarios para calibrar umbrales y pesos. (El perfil/feedback ya se sincronizan con la cuenta vía auth + Google, y los partidos agendados se persisten en la base de datos.)